

Go Back

Hardware

UNO Q

Tutorials

UNO Q User Manual

UNO Q as a Single-Board Computer

UNO Q Power Specifications

Flashing a New Image to the UNO Q

Connect to UNO Q via Secure Shell (SSH)

Connect UNO Q to the Arduino Cloud

Home / Hardware / UNO Q / UNO Q User Manual

UNO Q User Manual

Learn about the hardware and software features of the Arduino® UNO Q.

Author • Christopher Méndez Last revision • 10/28/2025

ON THIS PAGE

Overview

Hardware and Software Requirements	+
Product Overview	+
First Use	+
Onboard User Interface	+
USB-C Connector	
Pins	+
Communication	+
Wireless Connectivity	+
Support	+

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino UNO Q. With this user manual, you will learn how to set up, configure and use this Arduino board.



Arduino UNO Q

Hardware and Software Requirements

Hardware Requirements

UNO Q (1x)

USB-C® cable (1x)

USB-C multiport adapter (dongle) with external power delivery (1x)



You can use any USB-C dongle with external power delivery capabilities except for **Apple** ones.

Software Requirements

Arduino App Lab 0.1.23+



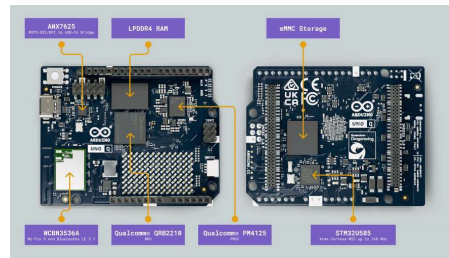
You can still use the **Arduino IDE 2+** to program only the microcontroller (MCU) side of your UNO Q.

Product Overview

The Arduino UNO Q unlocks a new level of performance for the Arduino ecosystem, blending robust computing power from Qualcomm's advanced QRB2210 Microprocessor (MPU) running a full Debian Linux OS with upstream support, and the real-time responsiveness of a dedicated STM32U585 Microcontroller (MCU) running Arduino sketches over

Board Architecture Overview

The Arduino UNO Q blends the high-performance Qualcomm® QRB2210 MPU, running a full Linux environment, with the real-time precision of the STMicroelectronics® STM32U585 (32-bit Arm® Cortex®-M33) MCU, all on a single, compact board. This mixed architecture delivers the power and responsiveness needed for AIoT, machine learning, and advanced automation applications.



UNO Q's main components

Here is an overview of the board's main components, as shown in the image above:

Microprocessor: The Qualcomm® QRB2210 is a quad-core Arm® Cortex®-A53 processor running at 2.0 GHz, equipped with an Adreno™ 702 GPU (845 MHz) for 3D graphics acceleration and dual ISPs supporting up to 25 MP at 30 fps. It runs Debian Linux OS with upstream support, making it well-suited for embedded vision and edge computing applications.

Microcontroller: The STM32U585 microcontroller features an Arm® Cortex®-M33 core running up to 160 MHz, with 2 MB of flash memory and 786 KB of SRAM. It runs the Zephyr OS, providing a secure and efficient platform for low-power embedded applications.

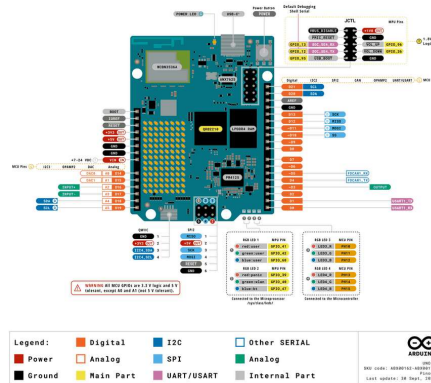
Wireless Connectivity: The WCBN3536A radio module provides dual-band Wi-Fi® 5 (2.4/5 GHz) and Bluetooth® 5.1 connectivity, both with onboard antennas for reliable wireless performance.

Memory: The board features 16 GB or 32 GB options of eMMC storage and 2 GB or 4 GB options of LPDDR4 RAM, delivering fast memory access and reliable storage for embedded applications.

Multimedia Codec: The ANX7625 multimedia codec enables video and audio output through the onboard USB-C connector, providing a high-speed interface for display and sound transmission in embedded applications.

Power Management: The UNO Q includes the Qualcomm® PM4145, a power management integrated circuit (PMIC) to meet the demands of always-connected IoT devices.

Pinout



UNO Q Simple pinout

The full pinout is available and downloadable as PDF from the link below:

[UNO Q full pinout](#)

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

[UNO Q datasheet](#)

Schematics

The complete schematics are available and downloadable as PDF from the link below:

[UNO Q schematics](#)

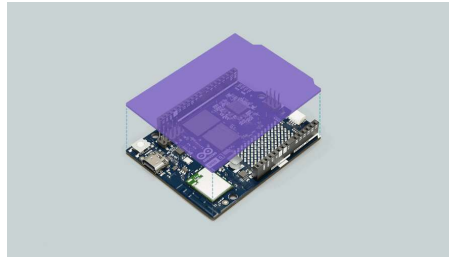
STEP Files

The complete STEP files are available and downloadable from the link below:

[UNO Q STEP files](#)

Form Factor

The Arduino UNO Q features the classic UNO form factor, ensuring compatibility with the wide range of UNO shields developed by us and the community over time.



UNO form factor

First Use

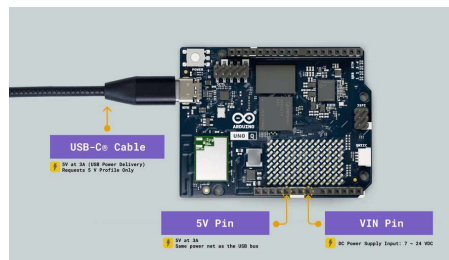
Powering the Board

The Arduino UNO Q can be powered by:

- A USB-C® cable providing 5 VDC 3 A (not included).

- An external +5 VDC power supply connected to 5V pin.

- An external +7-24 VDC power supply connected to VIN pin.



UNO Q power options

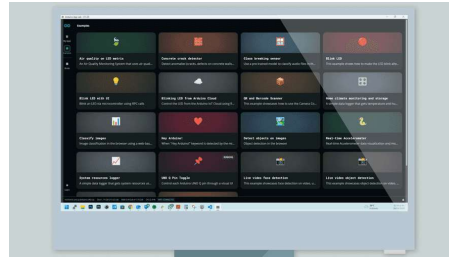


Click [here](#) to learn more about the UNO Q power specifications.

Install Arduino App Lab

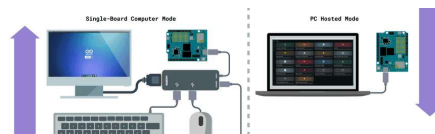
The **Arduino App Lab** is a unified development environment that extends the classic Arduino experience into the world of high-performance computing. Arduino App Lab lets you seamlessly combine Arduino sketches, Python scripts, and containerized Linux applications into a single workflow.

With code building blocks called Bricks, preconfigured AI models, and integrated orchestration, it reduces complexity while enabling you to create everything



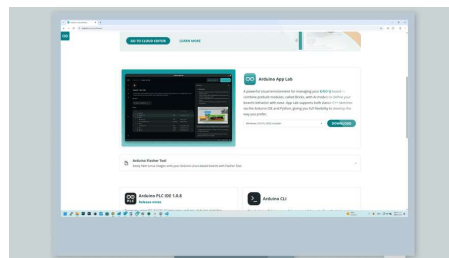
Arduino App Lab IDE

Arduino App Lab comes **pre-installed** on the UNO Q and can be used in Single-Board Computer (SBC) mode. We highly recommend the **4 GB of RAM** UNO Q variant for a better **standalone** experience.



SBC and PC hosted modes

To install it in your personal computer for a **PC Hosted** setup, go to the [software section](#) on our official website, scroll to Arduino App Lab and select your OS's respective variant.



Arduino App Lab Install

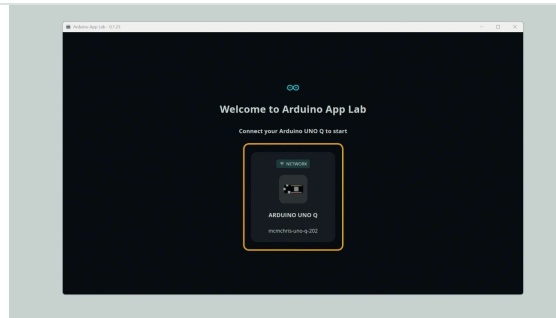
Network Mode

Even when you have set up your Arduino UNO Q as a **single-board computer**, you can access it remotely from your personal machine by using the Arduino App Lab desktop and the **Network Mode**. Both modes can be used simultaneously.

When you open Arduino App Lab, you will see your board listed with the "Network" tag.

Click on it and enter the Linux password to log in.

You will now have access to the board remotely.



With this method, you can access your UNO Q from any machine in your local network. This allows you to use Arduino App Lab as if you were connected directly to the board, where you can develop & run Apps in the same way as if it was connected via USB-C®.

Hello World Example

Let's program the UNO Q with the classic **Hello World** example typical of the Arduino ecosystem: the Blink sketch. We will use this example to verify that the board is correctly connected to the Arduino App Lab.

Prerequisites:

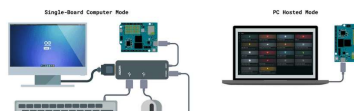
Arduino UNO Q

USB-C cable

Computer with internet access (PC-hosted mode)

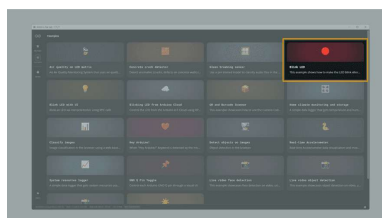
Monitor, keyboard, mouse, and USB-C dongle (single-board computer mode)

Connect the UNO Q to your PC (if you are not in single-board computer mode).



SBC mode and PC-Hosted setup

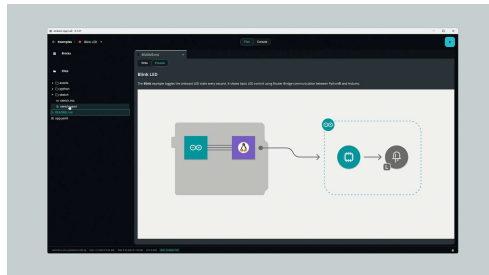
Open the Arduino App Lab, it opens in the **Examples** section.



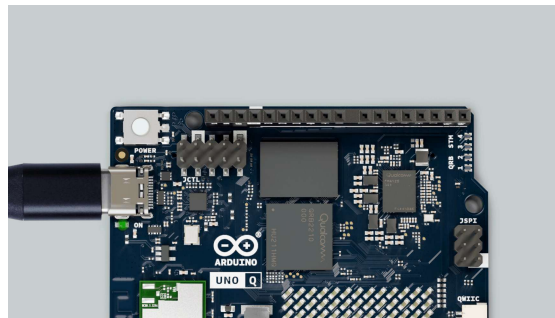
Arduino App Lab Landing Page

Open the **Blink LED** example (Read the example

Click on the **Run** button in the top right corner and wait for the app to be uploaded.



You should now see the red LED of the built-in RGB LED turning on for one second, then off for one second, repeatedly.



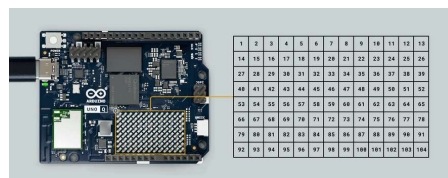
The LED controlled in this example is driven by the STM32 microcontroller through the Arduino sketch.

Onboard User Interface

The Arduino UNO Q offers a wide range of user interfaces, making interaction intuitive and straightforward.

LED Matrix

One of the board's key features is an 8×13 blue LED matrix that is managed by the STM32 microcontroller of the UNO Q.

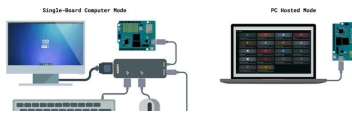


LED matrix

It is a versatile display you can use to show data, status indicators, icons, or even create simple animations and games.

Here is a list of basic examples for using the **LED matrix**. To test them, follow the steps below:

Connect the UNO Q to your PC (if you are not in single-board computer mode).



SBC mode and PC-Hosted setup

Open the Arduino App Lab, navigate to **My Apps** and click on **Create new app+**.



Create a new app

A new App must be created to test each of the examples below.

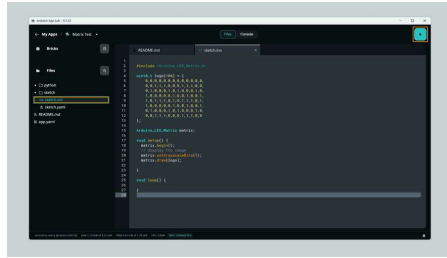
Image Drawing

This example is for drawing **custom frames** in the LED matrix, specifically the Arduino logo.

You can copy and paste the following example into the "sketch" part of your new App in the Arduino App Lab.

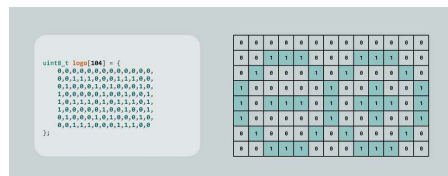
```

1  #include <Arduino_LED_Matrix.h>
2
3  uint8_t logo[104] = {
4      0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5      0,0,1,1,1,0,0,0,1,1,1,0,0,0,
6      0,1,0,0,0,1,0,1,0,0,0,1,0,
7      1,0,0,0,0,0,1,0,0,1,0,0,1,
8      1,0,1,1,1,0,1,0,1,1,1,0,1,
9      1,0,0,0,0,0,1,0,0,1,0,0,1,
10     0,1,0,0,0,1,0,1,0,0,0,1,0,
11     0,0,1,1,1,0,0,0,1,1,1,0,0,
12 };
13
14 Arduino_LED_Matrix matrix;
15
16 void setup() {
17     matrix.begin();
18     // display the image
19     matrix.setGrayscaleBits(1);
20     matrix.draw(logo);
21 }
22 }
23
24 void loop() {
25
26 }
```



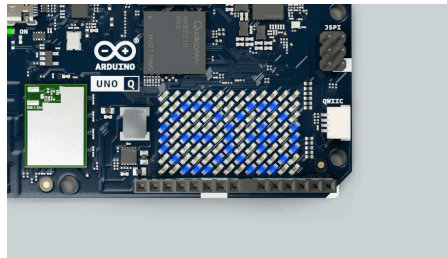
Example ready to be uploaded

You can create your own frame by creating an array following the matrix format (8x13) with 1's and 0's as in the example from above:



Matrix frame example

Execute the App by clicking on the **Run** button in the Arduino App Lab and you should see the LED matrix showing your frame:



LED Matrix example running

Dimmable LEDs

The LED matrix supports 8 levels of grayscale (3 bits) so you can manage the LED brightness individually.

You can set the brightness bits with the function

`setGrayscaleBits(bits)` as shown below:

```
1 matrix.setGrayscaleBits(3); // 3 bits r
```

As usual conversion tools to grayscale uses 256 levels (8 bits) so you can also use this range, and it will be automatically mapped.

```
1 matrix.setGrayscaleBits(8); // 8 bits r
```

This example is for showing the **supported grayscale** in the LED matrix.

You can copy and paste the following example into the "sketch" part of your new App in the Arduino App Lab.

```
1 #include <Arduino_LED_Matrix.h>
2
3 uint8_t shades[104] = {
4     0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
5     1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,
6     2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,
7     3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,3,
8     4,4,4,4,4,4,4,4,4,4,4,4,4,4,4,4,
9     5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,5,
10    6,6,6,6,6,6,6,6,6,6,6,6,6,6,6,6,
11    7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,7,
12 };
13
14 Arduino_LED_Matrix matrix;
15
16 void setup() {
17     matrix.begin();
18     // display the image
19     matrix.setGrayscaleBits(3);
20     matrix.draw(shades);
21 }
22
23
24 void loop() {
25
26 }
```

Execute the App by clicking on the **Run** button in the Arduino App Lab and you should see the LED matrix showing your frame:

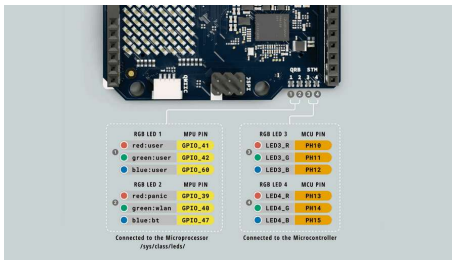


LED Matrix example running

RGB LEDs

The UNO R3 features 4x RGB LEDs. Two of them

microprocessor, and the other two by the STM32 microcontroller.



UNO Q RGB LEDs

MPU Controlled LEDs

LEDs #1 and #2 are controlled by the MPU.

There is a dedicated LED interface in our Linux OS for controlling these LEDs, they can be controlled via `/sys/class/leds` from the **Command Line**, using **SSH**, an **ADB** connection from your PC terminal or by using the Linux built-in terminal application when used in single-board computer mode:

```
1 echo 1 | sudo tee /sys/class/leds/red/trigger
2 echo 0 | sudo tee /sys/class/leds/red/trigger
```



The LED color segments are defined as follows:

LED 1:

- Red: red:user
- Green: green:user
- Blue: blue:user

LED 2:

- Red: red:panic
- Green: green:wlan
- Blue: blue:bt

 LED 2 is used to show system status: `PANIC`, `WLAN` and `BT`. But it can be controlled by the user.

You can also control these LEDs from a Python script as follows. Remember to **create a new App** inside Arduino App Lab and then copy and paste the script below in the python section of your App:

```

1  import time
2
3  LED1_R = "/sys/class/leds/red:user/br
4  LED1_G = "/sys/class/leds/green:user/
5  LED1_B = "/sys/class/leds/blue:user/b
6
7  LED2_R = "/sys/class/leds/red:panic/b
8  LED2_G = "/sys/class/leds/green:wlan/
9  LED2_B = "/sys/class/leds/blue:bt/bri
10
11 def set_led_brightness(led_file, valu
12     try:
13         with open(led_file, "w") as f
14             f.write(f"{value}\n")
15     except Exception as e:
16         print(f"Error writing to {led
17
18 def main():
19     # turn off all LEDs
20     set_led_brightness(LED1_R, 0)
21     set_led_brightness(LED1_G, 0)
22     set_led_brightness(LED1_B, 0)
23     set_led_brightness(LED2_R, 0)
24     set_led_brightness(LED2_G, 0)
25     set_led_brightness(LED2_B, 0)
26
27     while True:
28         #blink the LED 1 RED segment

```

MCU Controlled LEDs

LEDs #3 and #4 are controlled by the MCU.

They can be controlled by setting the state of their respective GPIOs using the `digitalWrite` function as usual.

To test them follow the steps below:

Connect the UNO Q to your PC (if you are not in single-board computer mode).

Open the Arduino App Lab, navigate to **My Apps** and click on **Create new app+**.

You can copy and paste the following example into the "sketch" part of your new App in the Arduino App Lab.

```
1 void setup(){
2   // Configure the pins as outputs
3   pinMode(LED3_R, OUTPUT);
4   pinMode(LED3_G, OUTPUT);
5   pinMode(LED3_B, OUTPUT);
6   // As they are active low, turn them
7   digitalWrite(LED3_R, HIGH);
8   digitalWrite(LED3_G, HIGH);
9   digitalWrite(LED3_B, HIGH);
10 }
11
12 void loop(){
13   digitalWrite(LED3_R, LOW); // Turn
14   digitalWrite(LED3_G, HIGH);
15   digitalWrite(LED3_B, HIGH);
16   delay(1000);
17   digitalWrite(LED3_R, HIGH);
18   digitalWrite(LED3_G, LOW); // Turn
19   digitalWrite(LED3_B, HIGH);
20   delay(1000);
21   digitalWrite(LED3_R, HIGH);
22   digitalWrite(LED3_G, HIGH);
23   digitalWrite(LED3_B, LOW); // Turn
24   delay(1000);
25 }
```



The LED color segments are defined as follows:

LED 3:

Red: LED3_R

Green: LED3_G

Blue: LED3_B

LED 4:

Red: LED4_R

Green: LED4_G

Blue: LED4_B

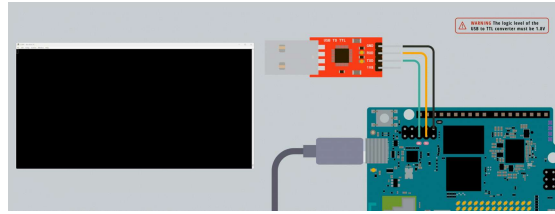


The RGB LEDs are active low, this means they turn ON with logic '0'.

Hardware Debug UART Interface

The UNO Q provides a dedicated low-level UART interface for debugging and system diagnostics. This interface connects directly to the SoC's main console

environment before network services like SSH or ADB are available.



This interface is available through the JCTL connector on the UNO Q. Refer to the [pinout](#) section for details, and follow the wiring example above to access it.

WARNING: This interface operates at **1.8 V logic** levels and must be used with a compatible USB-to-TTL converter to avoid hardware damage.

Prerequisites

1.8V USB to TTL converter (e.g., DSD Tech SH-U09C5)

USB-C cable to power the UNO Q

Serial Terminal (e.g., [Tera Term](#))

UART Parameters:

Baud rate: 115200 bps

Logic level: 1.8 V

This console provides access to low-level system messages printed by the bootloaders (e.g., SPL and U-Boot), which are not visible through SSH or other high-level interfaces. For example, it allows capturing logs related to power delivery negotiation or hardware initialization during early boot stages, providing information that is otherwise inaccessible.

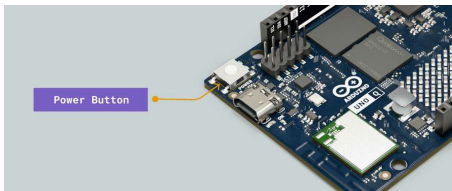
It can be used to log in to the board and interact with the system through the shell. Use the **Linux credentials** configured during the board setup process to authenticate.



Shell Log in

Power Button

The UNO Q features a power button that can be used to reboot the board.



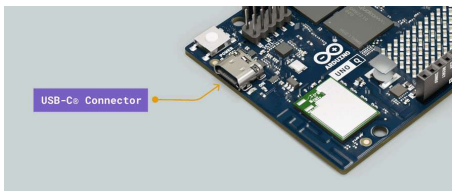
UNO Q power button

Long press: the board's Linux part is rebooted when the button is pressed for **5+** seconds.

 You do not need to press the power button for the board to power up, it boots automatically after being powered.

USB-C Connector

The UNO Q features a USB-C connector that can be used for much more than just programming and powering the board.



USB-C connector

Below is a table with the main features of the USB-C connector that expands the UNO Q capabilities.

Feature	Description
USB Power (Sink)	5 VDC 3 A (15 W)
USB Standard	USB 3.1 Gen 1 (5 Gb/s)
Display over USB-C	DisplayPort

By using a USB-C dongle (adapter/hub) you can also leverage the following features:

Feature	Description
Video Output	HDMI support
Video Input	USB camera support
Audio	USB or 3.5mm headset (speaker + microphone)

Feature	Description
HID	USB keyboard/mouse and other HID devices
Storage	External microSD card or USB drive support

Pins

The UNO Q is equipped with two connector types: the **classic UNO-style headers** on the top, designed for prototyping and debugging, guaranteeing full compatibility with Arduino UNO Shields, and the **high-speed header connectors** on the bottom, purpose-built for integration with UNO Q carriers.

Digital Pins

The UNO Q has 47x digital pins controlled by the **STM32 microcontroller**, 22x of them exposed through the UNO-styled connector and 25x exposed through JMISC connector mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PB7	D0 / RX	GPIO / UART RX
PB6	D1 / TX	GPIO / UART TX
PB3	D2	GPIO
PB0	D3	GPIO / OPAMP OUT
PA12	D4 / FDCAN1_TX	GPIO / CAN Bus TX
PA11	D5 / FDCAN1_RX	GPIO / CAN Bus RX
PB1	D6	GPIO
PB2	D7	GPIO
PB4	D8	GPIO
PB8	D9	GPIO
PB9	D10 / SS	GPIO / SPI SS
PB15	D11 / MOSI	GPIO / SPI MOSI
PB14	D12 / MISO	GPIO / SPI MISO
PB13	D13 / SCK	GPIO / SPI SCK
PA4	D14 / DAC0	GPIO / ADC / DAC
PA5	D15 / DAC1	GPIO / ADC / DAC
PA6	D16	GPIO / ADC / OPAMP IN +
PA7	D17	GPIO / ADC / OPAMP IN -

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PC1	D18 / SDA2	GPIO / ADC / I2C SDA
PC0	D19 / SCL2	GPIO / ADC / I2C SCL
PB11	D20 / SDA	GPIO / I2C SDA
PB10	D21 / SCL	GPIO / I2C SCL

 Notice that pins D14 to D19 also have analog capabilities.

The digital pins of the UNO Q can be used as inputs or outputs through the built-in functions of the Arduino programming language.

The configuration of a digital pin is done in the `setup()` function with the built-in function `pinMode()` as shown below:

```
1 // Pin configured as an input
2 pinMode(pin, INPUT);
3 // Pin configured as an output
4 pinMode(pin, OUTPUT);
5 // Pin configured as an input, internal
6 pinMode(pin, INPUT_PULLUP);
```

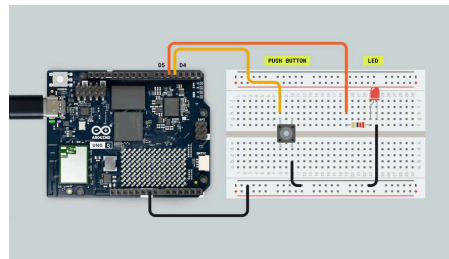
The state of a digital pin, configured as an input, can be read using the built-in function `digitalRead()` as shown below:

```
1 // Read pin state, store value in a sta
2 state = digitalRead(pin);
```

The state of a digital pin, configured as an output, can be changed using the built-in function `digitalWrite()` as shown below:

```
1 // Set pin on
2 digitalWrite(pin, HIGH);
3 // Set pin off
4 digitalWrite(pin, LOW);
```

The example code shown below uses digital pin `D5` to control an LED and reads the state of a button connected to digital pin `D4`:



Digital I/O example wiring

Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



Create a new app

```
1 #include <Arduino_RouterBridge.h>
2 // Define button and LED pin
3 int buttonPin = D4;
4 int ledPin = D5;
5
6 // Variable to store the button state
7 int buttonState = 0;
8
9 void setup() {
10   // Configure button and LED pins
11   pinMode(buttonPin, INPUT_PULLUP);
12   pinMode(ledPin, OUTPUT);
13
14   // Initialize Serial communication
15   Monitor.begin();
16 }
17
18 void loop() {
19   // Read the state of the button
20   buttonState = digitalRead(buttonPin);
21
22   // If the button is pressed, turn on the LED
23   if (buttonState == LOW) {
24     digitalWrite(ledPin, HIGH);
25     Monitor.println("- Button is pressed");
26   } else {
27     // If the button is not pressed, turn off the LED
28     digitalWrite(ledPin, LOW);
29     Monitor.println("- Button is not pressed");
30   }
```

The UNO Q features the well-known analog pins in the **JANALOG** connector; more details below:

Analog to Digital Converter (ADC)

In the **JANALOG** connector the UNO Q has 6x 14-bit ADC pins mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PA4	A0	GPIO / ADC / DAC
PA5	A1	GPIO / ADC / DAC
PA6	A2	GPIO / ADC / OPAMP IN +
PA7	A3	GPIO / ADC / OPAMP IN -
PC1	A4	GPIO / ADC / I2C SDA
PC0	A5	GPIO / ADC / I2C SCL

Analog input pins can be used through the built-in functions of the Arduino programming language.

The UNO Q ADC **resolution** can be configured between 14, 12, 10, or 8 bits by using the `analogReadResolution(bits)` function:

```
1 // ADC resolution set to 14-bit (0 to 1
2   analogReadResolution(14);
```

The default ADC **voltage reference** is 3.3V and can be changed by software using the function `analogReference()` with the following arguments:

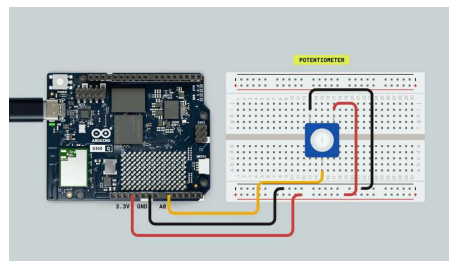
Analog Voltage Reference (V _{REF+})	Argument	Source
1.5 V	AR_INTERNAL1V5	Internal
1.8 V	AR_INTERNAL1V8	Internal
2.048 V	AR_INTERNAL2V05	Internal
2.5 V	AR_INTERNAL2V5	Internal
2 V ~ VDD	AR_EXTERNAL	External

An external voltage reference can be provided through the **AREF** pin when `AR_EXTERNAL`

To set a different analog reference from the default one, see the following example:

```
1 analogReference(AR_INTERNAL2V5);
```

The example code shown below reads the analog input value from a potentiometer connected to `A0` and displays it on the Serial Monitor. To understand how to properly connect a potentiometer to the UNO Q, take the following image as a reference:



ADC input example wiring

Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



Create a new app

```
1 #include <Arduino_RouterBridge.h>
2
3 int sensorPin = A0; // select the in
4
5 int sensorValue = 0; // variable to s
6
7 void setup() {
8   Monitor.begin();
9 }
10
11 void loop() {
12   // read the value from the sensor:
13   sensorValue = analogRead(sensorPin);
14
15   Monitor.println(sensorValue);
16   delay(100);
17 }
```

The UNO Q has two DAC outputs, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PA4	DAC0	GPIO / ADC / DAC
PA5	DAC1	GPIO / ADC / DAC

The digital-to-analog converters of the UNO Q can be used to output analog voltages through the built-in functions of the Arduino programming language.

The DAC output resolution can be configured from 8 to 12 bits using the `analogWriteResolution()` function as follows:

```
1 // DAC resolution set to 12-bit (8 to 12 bits)
2 analogWriteResolution(12); // enter the resolution
```

To output an analog voltage value through a DAC pin, use the `analogWrite()` function with the DAC channel as an argument. See the example below:

```
1 analogWrite(DAC0, value); // the value to output
```

 If a normal GPIO is passed to the `analogWrite()` function, the output will be a PWM signal.

The following sketch will create a **60 Hz sine wave** signal in the `A0/DAC0` UNO Q pin:

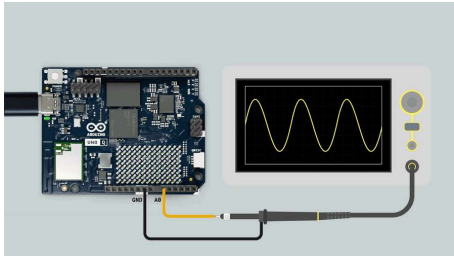
Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



Create a new app

```
1  const float freq = 60.0f;
2  const int    N    = 256;      // 256 samples
3  const uint32_t Ts_us = (uint32_t)110000; // 110µs
4
5  uint16_t lut[N]; // store the sine wave
6
7  void setup() {
8      analogWriteResolution(12);
9
10     for (int i = 0; i < N; ++i){
11         lut[i] = 2048 + (1000.0 * sin(2
12     }
13 }
14 }
15
16 void loop() {
17     static uint32_t t_next = micros();
18     for (int i = 0; i < N; ++i) {
19         analogWrite(DAC0, lut[i]); // output
20         t_next += Ts_us;
21         while ((int32_t)(micros() - t_next) < 0) {}
22     }
23 }
```

The DAC output should look like the image below:



Analog Sine Wave DAC output

PWM Pins

The UNO Q has 6x PWM (Pulse Width Modulation) pins, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping	Pin Functionality
PB0	D3	GPIO / OPAMP OUT / PWM
PA11	D5 / FDCAN1_RX	GPIO / CAN Bus RX / PWM
PB1	D6	GPIO / PWM
PB8	D9	GPIO / PWM
PB9	D10 / SS	GPIO / SPI SS / PWM

This functionality can be used with the built-in function `analogWrite()` as shown below:

```
1 analogWrite(pin, value);
```

By default, the output resolution is **8 bits**, so the output value should be between 0 and 255. To set a greater resolution, do it using the built-in function `analogWriteResolution` as shown below:

```
1 // PWM resolution set to 10-bit (0 to 4095)
2 analogWriteResolution(10);
```

Here is an example of how to create a variable duty-cycle PWM signal:

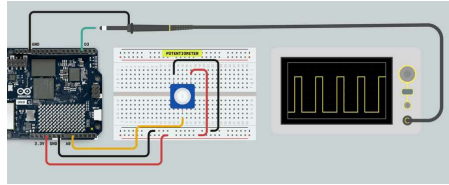
Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



Create a new app

```
1 const int analogInPin = A0; // Analog
2 const int pwmOutPin = D3;   // PWM ou
3
4 int sensorValue = 0; // value read fr
5 int outputValue = 0; // value output
6
7 void setup() {
8   // Define the PWM output resolution
9   analogWriteResolution(10); // 0 - 1
10  analogReadResolution(14);  // 0 - 1
11 }
12
13 void loop() {
14   // read the analog in value:
15   sensorValue = analogRead(analogInPin);
16   // map it to the range of the analog
17   outputValue = map(sensorValue, 0, 16383, 0, 255);
18   // change the analog out value:
19   analogWrite(pwmOutPin, outputValue);
20
21   // wait 2 milliseconds before the ne
22   // to settle after the last reading:
23   delay(2);
```

Now you can control the PWM signal duty-cycle by turning the potentiometer.



PWM output signal using the PWM

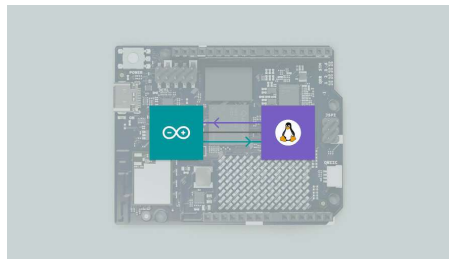
i PWM frequency is fixed to 500 Hz.

Communication

This section of the user manual covers the different communication protocols that are supported by the Arduino UNO Q.

Bridge - Remote Procedure Call (RPC) Library

The Arduino UNO Q uses RPC (Remote Procedure Call) to exchange data between the Linux (Qualcomm MPU) side and the real-time STM32 MCU. This mechanism allows functions running on one processor to be invoked transparently from the other, as if they were local calls.



UNO Q RPC

Overview

The `Bridge` library provides a communication layer built on top of the `Arduino_RPClite` framework. It manages bidirectional RPC traffic between the MPU and MCU, handling method binding, request forwarding, and asynchronous responses.

MPU side (Qualcomm QRB, Linux): Runs higher-level services and can remotely invoke

MCU side (STM32, Zephyr RTOS): Handles time-critical tasks and exposes functions to the MPU via RPC.

Core Components

BridgeClass

Main class managing RPC clients and servers.
Provides methods to:

Initialize the bridge (`begin()`)

Call remote procedures (`call()`)

Notify without waiting for a response (`notify()`)

Expose local functions for remote execution (`provide()` , `provide_safe()`)

Process incoming requests (`update()`)

RpcResult

Helper class representing the result of a remote call. It waits for the response, extracts the return value, and propagates error codes if needed.

Threading and Safety

The bridge uses Zephyr mutexes (`k_mutex`) to guarantee safe concurrent access when reading/writing over the transport. Updates are handled by a background thread that continuously polls for requests.

Usage Example

This example shows the **Linux side (Qualcomm QRB)** toggling an LED on the **MCU (STM32)** by calling a remote function over the Bridge.

Create a new App in the Arduino App Lab, then copy and paste the example below in the "Python" and "sketch" parts of your new App respectively.



Create a new app

1 Linux (QRB) example to call a remote MCU function

This Python script runs on the QRB and calls an MCU-exposed RPC named `set_led_state` once per second:

```

1  # main.py (QRB side)
2  from arduino.app_utils import
3  import time
4
5  led_state = False
6
7  def loop():
8      global led_state
9      time.sleep(1)
10     led_state = not led_state
11     Bridge.call("set_led_state", led_state)
12
13 App.run(user_loop=loop)

```

This sends a boolean to the MCU every second using

```
Bridge.call("set_led_state", <bool>)
```

2 MCU (STM32) setup to include the Bridge and start it

This sketch includes the Bridge library and configures the LED pin.

```

1  #include "Arduino_RouterBridge.h"
2
3  void setup() {
4      pinMode(LED_BUILTIN, OUTPUT);
5
6      Bridge.begin();
7      Bridge.provide("set_led_state", set_led_state);
8  }
9
10 void loop() {
11 }
12
13 void set_led_state(bool state) {
14     // LOW state means LED is on
15     digitalWrite(LED_BUILTIN, !state);
16 }

```

This registers the local MCU function

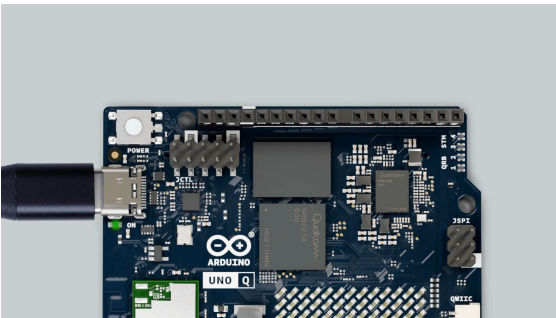
`set_led_state` as an RPC service named `"set_led_state"`, so that the Linux (QRB)

side can call it remotely as if it were a local function using

```
Bridge.provide("set_led_state", set_led_state);
```

i You can do the same the other way around, Python functions can be provided to the MCU sketch to be used locally.

After pasting the Python script into your App's Python file and the Arduino code to the sketch, you can run the App and observe LED #3 blinking in red every second.



 There are more advanced methods in the Bridge RPC library that you can discover by testing our different built-in examples inside Arduino App Lab.

SPI

The UNO Q supports SPI communication, which allows data transmission between the board and other SPI-compatible devices.

The pins used in the UNO Q for the SPI communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
PB9	SS / D10
PB15	MOSI / D11
PB14	MISO / D12
PB13	SCK / D13

Please, refer to the [board pinout section](#) of the user manual to locate them on the board.

Include the `SPI` library at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
1 #include <SPI.h>
```

In the `setup()` function, initialize the SPI library, define and configure the chip select (`SS`) pin:

```
1 #define SS D10
2
3 void setup() {
4   // Set the chip select pin as output
5   pinMode(SS, OUTPUT);
6
7   // Pull the SS pin HIGH to unselect
8   digitalWrite(SS, HIGH);
9
10  // Initialize the SPI communication
11  SPI.begin();
12 }
```

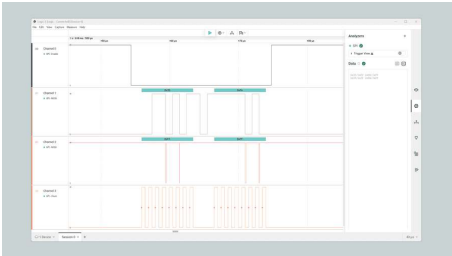
To transmit data to an SPI-compatible device, you can use the commands used in the following example:

Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



Create a new app

```
1 #include <SPI.h>
2
3 #define SS D10
4
5 void setup() {
6   // Set the chip select pin as output
7   pinMode(SS, OUTPUT);
8
9   // Pull the SS pin HIGH to unselect
10  digitalWrite(SS, HIGH);
11
12  // Initialize the SPI communication
13  SPI.begin();
14 }
15
16 void loop() {
17   // Replace with the target device's
18   byte address = 0x35;
19   // Replace with the value to send
20   byte value = 0xFA;
21   // Pull the SS pin LOW to select the device
22   digitalWrite(SS, LOW);
23   // Send the address
24   SPI.transfer(address);
25   // Send the value
26   SPI.transfer(value);
27   // Pull the SS pin HIGH to unselect
28   digitalWrite(SS, HIGH);
29 }
```



SPI data stream

I2C

The UNO Q supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. The pins used in the UNO Q for the I2C communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping (Wire)	Microcontroller Pin	Arduino Pin Mapping (Wire1)
PB10	SCL / D21	PD12	I2C4_SCL (Qwiic)
PB11	SDA / D20	PD13	I2C4_SDA (Qwiic)

Please, refer to the [board pinout section](#) of the user manual to locate them on the board.

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
1 #include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

```
1 // Initialize the I2C communication
2 Wire.begin(); // I2C in UNO-style header
3 // or
4 Wire1.begin(); // I2C in Qwiic connector
```

To transmit data to an I2C-compatible device, you can use the commands used in the following example:

Create a new App in the Arduino App Lab, then copy

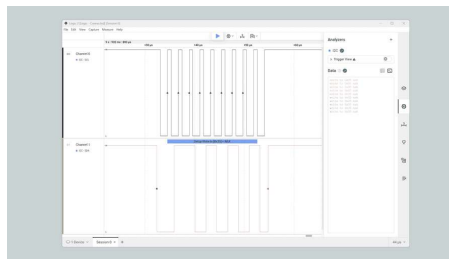
your new App.



Create a new app

```
1  #include <Wire.h>
2
3  void setup() {
4    // Initialize the I2C communication
5    Wire.begin();
6  }
7
8  void loop() {
9    // Replace with the target device's
10   byte deviceAddress = 0x35;
11   // Replace with the appropriate inst
12   byte instruction = 0x00;
13   // Replace with the value to send
14   byte value = 0xFA;
15   // Begin transmission to the target
16   Wire.beginTransmission(deviceAddress);
17   // Send the instruction byte
18   Wire.write(instruction);
19   // Send the value
20   Wire.write(value);
21   // End transmission
22   Wire.endTransmission();
23
24   delay(2000);
25 }
```

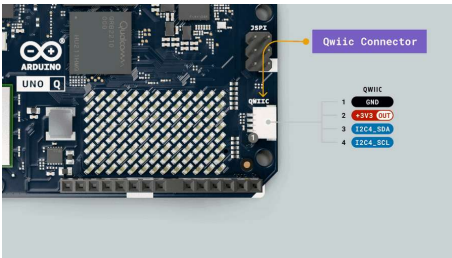
The example code above should output this:



I2C data stream

Qwiic

The Arduino UNO Q features an onboard Qwiic connector that provides a simple, tool-free solution for connecting I²C devices. The Qwiic ecosystem, developed by SparkFun Electronics, has become an industry standard for rapid prototyping with I²C



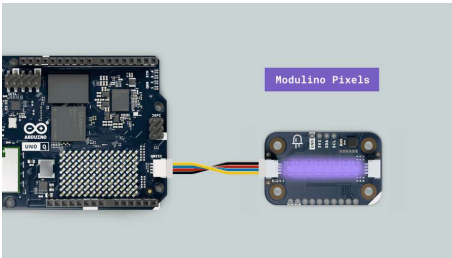
I2C Qwiic connector

The Qwiic system’s key advantages include:

- Plug-and-play connectivity:** No breadboards, jumper wires, or soldering required
- Polarized connectors:** Prevents accidental reverse connections
- Daisy-chain capability:** Connect multiple devices in series
- Built-in pull-up resistors:** No external resistors needed
- Standard pinout:** Compatible across all Qwiic ecosystem devices

i The Qwiic connector on the UNO Q is connected to the secondary I2C bus (I2C4), which uses the `Wire1` object rather than the `Wire` object. Please note that the Qwiic connector is 3.3 V only.

The Qwiic connector allows you to interface our Modulino nodes for developing soldering-free projects.



Modulino nodes

You can check our [Modulino family](#) where you will find a variety of **sensors** and **actuators** to expand your projects.

UART

The pins used in the UNO Q for the UART communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
---------------------	---------------------

Microcontroller Pin	Arduino Pin Mapping
PB7	USART1_RX / D0

Please, refer to the [board pinout section](#) of the user manual to locate them on the board.

To begin with UART communication, you will need to configure it first. In the `setup()` function, set the baud rate (bits per second):

```
1 // Start UART communication at 115200 b
2 Serial.begin(115200);
```

To transmit data to another device via UART, you can use the `write()` function:

```
1 // Transmit the string "Hello UNO Q"
2 Serial.write("Hello UNO Q");
3 Serial.write("\r\n"); // new line
```

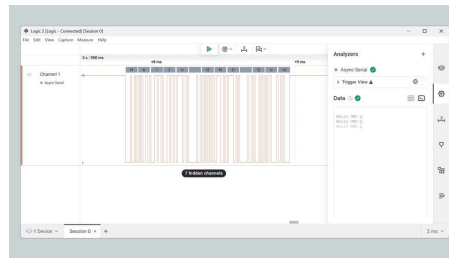
You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
1 // Transmit the string "Hello UNO Q"
2 Serial.print("Hello UNO Q");
3
4 // Transmit the string "Hello UNO Q" fo
5 Serial.println("Hello UNO Q");
```

To test the UART transmit method use the following example, remember to create a new App in the Arduino App Lab, then copy and paste the example below:

```
1 void setup() {  
2   // Initialize the hardware UART at 1  
3   Serial.begin(115200);  
4 }  
5  
6 void loop() {  
7   // Transmit the string "Hello UNO Q"  
8   Serial.println("Hello UNO Q");  
9   delay(1000);  
10 }
```

You should get the following in the **TX** and **RX** pins of your UNO Q board, I am using a logic analyzer to capture the data:



UART transmission

To read incoming data, you can use a `while()` loop to continuously check for available data and read individual characters. The code shown below stores the incoming characters in a String variable and processes the data when a line-ending character is received:

```
1 String incoming = "";
2
3 void setup() {
4   // Initialize the hardware UART at 1
5   Serial.begin(115200);
6 }
7
8 void loop() {
9   while (Serial.available()) {
10    char c = Serial.read();
11
12    if (c == '\n') {
13      // Echo the buffered message and
14      Serial.println(incoming);
15
16      // Clear for the next message
17      incoming = "";
18    } else {
19      incoming += c;
20    }
21  }
22 }
```

With this example the UNO Q will send back whatever it receives on the UART.

From Serial to Monitor

Because of the UNO Q's architecture, using `Serial` does not display data in the Arduino App Lab Serial Monitor as you might expect.

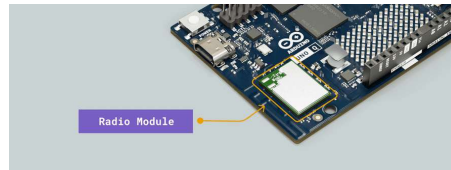
To make debugging just as easy as on other Arduino boards, we provide the `Monitor` object, which you can use to print debugging messages, sensor readings, or any other information directly to the Serial Monitor.

You can do exactly the same, but with a minor prerequisite; including the `Arduino_RouterBridge` library in your sketch:

```
1 #include <Arduino_RouterBridge.h>
2
3 void setup() {
4   // Initialize the Monitor
5   Monitor.begin();
6 }
7
8 void loop() {
9   // Transmit the string "Hello UNO Q"
10  Monitor.println("Hello UNO Q");
11  delay(1000);
12 }
```

Wireless Connectivity

The UNO Q features the WCBN3536A radio module that provides dual-band Wi-Fi® 5 (2.4/5 GHz) and Bluetooth® 5.1 to the board. This allows seamless wireless connectivity for both IoT and peripheral communication.



Radio Module

Whether connecting to a local network, uploading data to the cloud, or communicating with Bluetooth-enabled devices such as smartphones and sensors, the UNO Q offers flexible and reliable options for your projects.

Wi-Fi®

Wi-Fi connectivity on the UNO Q allows the board to connect to local networks or the internet to access online services, perform software updates, and communicate with remote servers. Additionally, Wi-Fi can be configured to share its internet connection with the onboard microcontroller, allowing both systems to stay connected without additional network hardware.

From the Microprocessor

If you followed the Arduino App Lab first set up, you should be already connected to the internet. However, here is a brief explanation of how to do it manually.

To **connect** the UNO Q to the internet, simply go to the upper-right corner and click on the network icon. Then, search for available Wi-Fi® networks and select one.



Connect to the Wi-Fi network

Or run the following command in the terminal:

```
1 sudo nmcli d wifi connect <SSID> passwo
```

To **disconnect** the UNO Q from the current Wi-Fi network, go to the same place where you enabled it before, click on the network icon in the upper-right corner, and then click on "Disconnect".

Or run the following command in the terminal:

```
1 sudo nmcli d disconnect wlan0
```

i `wlan0` is the typical name of the Wi-Fi interface, you can verify yours running `nmcli device` in the terminal.

If you want to forget the saved network so it doesn't auto-connect again, you can also run:

```
1 sudo nmcli connection delete <SSID>
```

From the Microcontroller

Since the radio module is connected to the Qualcomm microprocessor, we need the **Bridge** to expose the connectivity to the microcontroller.

The following example gets the UTC time using TCP over socket RPC calls and prints it in the Serial Monitor:

Create a new App in the Arduino App Lab, then copy and paste the example below in the "sketch" part of your new App.



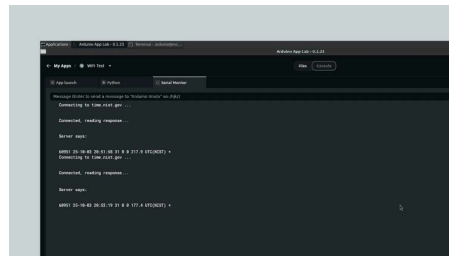
Create a new app

```

1  #include <Arduino_RouterBridge.h>
2
3  BridgeTCPClient<> client(Bridge);
4
5  void setup() {
6      if (!Bridge.begin()) {
7          while (true) {}
8      }
9      if (!Monitor.begin()) {
10         while (true) {}
11     }
12
13     Monitor.println("TCP Daytime Demo s
14 }
15
16 void loop() {
17     Monitor.println("\nConnecting to ti
18
19     if (client.connect("time.nist.gov",
20         Monitor.println("Connection faile
21         delay(5000);
22         return;
23     }
24
25     Monitor.println("Connected, reading
26     String line;
27     while (client.connected() || client
28         if (client.available()) {

```

Once running, open the Arduino App Lab Serial Monitor and you will see the time and date retrieved from the `time.nist.gov` server.

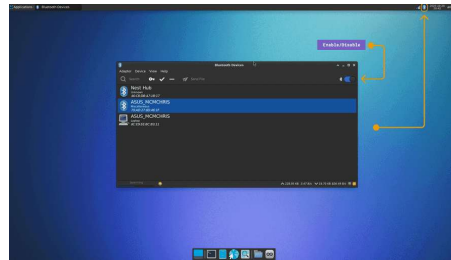


Wi-Fi example

Bluetooth®

Bluetooth® connectivity allows the UNO Q to communicate with nearby devices such as smartphones, computers, or sensors. It can be used for data exchange, remote control, or connecting to Bluetooth® peripherals like keyboards, headsets and serial devices. Depending on the setup, the UNO Q can act as either a Bluetooth® peripheral or host, enabling flexible short-range communication for various applications.

upper-right Bluetooth icon to manage it.



Bluetooth Manager

From here, you can do the following:

- Scan for devices
- Pair to a specific device
- Trust devices
- Send files
- Turn on or off the Bluetooth connection

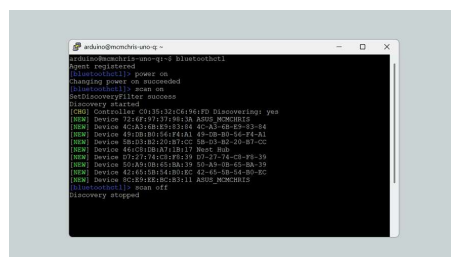
You can also manage the Bluetooth connection from the terminal by using `bluetoothctl` as follows:

```
1 bluetoothctl power on # turn on Bluetooth
2 bluetoothctl power off # turn off Bluetooth
```

You can enter the Bluetooth manager prompt by running `bluetoothctl` and inside you can run specific commands:

```
1 power on # turn on Bluetooth
2 power off # turn off Bluetooth
3 scan on # start searching for nearby Bluetooth devices
4 scan off # stop searching for devices
5 connect <MAC_ADDRESS> # pair to the device
```

Here is an example of how it looks like to search for Bluetooth devices from the terminal:



Support

If you encounter any issues or have questions while working with the Arduino UNO Q, we provide various support resources to help you find answers and solutions.

Help Center

Explore our [Help Center](#), which offers a comprehensive collection of articles and guides for the UNO Q. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[UNO Q Help Center page](#)

Forum

Join our community forum to connect with other UNO Q users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the UNO Q.

[UNO Q category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We are happy to help you with any issues or inquiries about the UNO Q.

[Contact us page](#)

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

